

CS598 HPC. Homework 1

Due: 9/2/26, 8 PM.

Q1. Your task is to build high performance *serial* C code to evaluate the matrix-matrix product, $C = AB$ for matrices $A \in \mathbb{R}^{L \times K}$ and $B \in \mathbb{R}^{K \times M}$. (Note that the task is *building*—not simply providing a fast code as an end-product.)

Assume that these are full matrices, with A and B stored as one-dimensional vectors having, respectively, LK and KM values, contiguously packed.¹

Your code should handle $K \neq L \neq M$, but you may assume that they are all multiples of 4 so that you can avoid loop-unrolling tail-outs, etc.

The code development should be for the **eng-instruction** partition on the campus cluster.

Steps:

1. Use matlab or octave to run a battery of tests on your target platform to identify a reasonable performance ceiling for a range of N -values, where $K = L = M := N$.

Matlab code is provided in `hw1.mat1.tar` that generates the type of graph we are interested in. **Note**, crucially, that this code divides the performance by the number of cores, `ncore`, as our C code will be for a *single core* only. (Later, we'll use MPI to realize P -fold parallelism...) Matlab/octave leverage multiple cores in `mxm` to gain maximum performance because it is the key kernel for all of linear algebra. When you run this baseline code, you should check (e.g., “top” on linux systems) to see how many cores are executing or what percentage ($> 100\%$) is being used—this is an estimate of the number of cores employed in your baseline matlab tests. If it is 8, then you are set, because the given code uses `ncore=8`, which is the value used for my Mac M1. *You can alternatively force matlab to use a single core—this is not so easy in octave, however.*

- What is your peak GFLOPS, and for what value of N is it realized?
 - Plot GFLOPS vs. N for the given matlab/octave code on your platform.
2. Write the following (with help from chatgpt or claude)
 - A C kernel to compute $C = AB$, where A , B , and C are one dimensional arrays (both in the calling routine and in the kernel).
 - A makefile
 - A driver that loops through a range of N , where $K = L = M := N$ (i.e., we test only for square `mxm`'s), similar to the matlab code.
 3. This will be your First Iteration.
 - What is your peak GFLOPS, and for what value of N is it realized?
 - Plot GFLOPS vs. N for this first iteration.
 - Paste your makefile and kernel on the same page as the graph.
 4. Work with chatgpt to iterate on the `mxm` kernel *and* the `makefile` until you reach at least 2/3s of the matlab/octave performance.
 5. For each iteration, set up a new page like the First Iteration.
 - What is your peak GFLOPS, and for what value of N is it realized?
 - Plot GFLOPS vs. N .
 - Paste the makefile and kernel source on this page.
 - Make a note of what changed between iterations.(Note: no need to present *each iteration*, just those that yield significant performance upticks.)
 6. Summarize your results.
 - Plot your best results along with the matlab results on a single graph.
 - What are the $N_{\frac{1}{2}}$ values for your C code and for the matlab code?
 - How do your results compare to Iteration 1 ?
 - Comment about results obtained, process, lessons learned, etc.

¹This assumption is not built into the LaPACK library `dgemm`, but it would be built into any focused HPC application, when possible.

Q2. Here, you will build an essential tool for analyzing parallel performance, namely a *point-to-point* ping-pong test. The ping-pong test provides a measure of round-trip message times between ranks q and p .

Write your codes using the “MPI-free” wrapped routines in the `msg_passing.files.tar` file that is tied to Lecture 1.

Your test will measure the time between ranks $q = 0$ and $p = 0, 1, 2, \dots, M$, where $M \leq P - 1$ is the maximum rank to be tested. (I generally choose $M = 512$, but this might not always be sufficient. The only reason to use $M < P - 1$ is if the test takes too long.)

Test for a range of message sizes (in 64-bit words) given by the pseudo-geometric progression below:

```
int n=0;
for (j=0; j<500; ++j) {
    nwds = (int)( nwds+1)*1.016 );
    byte = 8*nwds;

    nloop = msg_vol/((double)nwds+2);
    nloop = min(nloop,1000);
    nloop = max(nloop,20);

    /* call your ping-pong test here, which loops from 0 to nloop-1 */
}
```

This progression tests every message size $m = nwds$ for $m = 1, \dots, 64$. The geometric progression takes over when $m > 64$.

2a. On a single loglog plot, graph the 1/2-round-trip message times for each value of $p \in [0, P - 1]$ where the number of ranks is P_1 .

- What is the observed message latency (seconds)?
- What is the observed inverse bandwidth (seconds-per-word) as m grows large?
- What is the observed m_2 value, where $t(m_2) = 2t(1)$?
- What is the observed value of m where the eager limit is reached?

2b. Repeat **2a** using a larger number of MPI ranks, P_2 . (For **eng-instruction**, I recommend one node, with 64 and 128 ranks. Then, use two nodes, test 128, 192, and 256 ranks.

Technical Notes.

- Be sure to *warm start* your timer by first calling the ping-pong test outside the testing loop.
- Unlike the 2018 Eijkhout paper, here we are trying to measure *raw* MPI performance (round-trip message time / 2) as a function of message size, m . Therefore, we prefer to have *cached data* for this test, which means that we will send from the same buffer on each test and receive to the same receive buffer (different from send buffer). One could also run the test with noncached data, which would be a different measurement.
- Use `isend()` and `irecv()`, coupled with `msgwait`, which are the nonblocking send/receive options.
- Use `msg_wtime` to measure the time. The idea behind multiple loops is to make sure that the measured event takes longer than the clock resolution provided by `mpi_wtime`.

Example Solution. Here is an example Solution for Q1.

Below you can see initial and final makefiles and source code. (Sorry—I've shortcut some of the process—there were about 5 iterations to get to this stage.)

Initial

```
CC      = gcc
CFLAGS  = -O3 -march=native -std=c99 -Wall -Wextra
OBJJS   = bench.o dgemm_kernel.o
bench:  $(OBJJS)
$(CC) $(CFLAGS) -o bench $(OBJJS)
clean:
rm -f *.o bench
```

Final

```
CC = gcc
CFLAGS_COMMON = -std=c99 -Wall -Wextra
CFLAGS_bench = \
    $(CFLAGS_COMMON) \
    -O2
CFLAGS_dgemm_kernel = -O3 -ffast-math -fno-unroll-loops
OBJJS = \
    bench.o \
    dgemm_kernel.o
ASM = \
    bench.s \
    dgemm_kernel.s
all: bench $(ASM)
bench: $(OBJJS)
$(CC) -o $@ $(OBJJS)
bench.o: bench.c
$(CC) $(CFLAGS_bench) -c $< -o $@
dgemm_kernel.o: dgemm_kernel.c
$(CC) $(CFLAGS_dgemm_kernel) -c $< -o $@
bench.s: bench.c
$(CC) $(CFLAGS_bench) -S $< -o $@
dgemm_kernel.s: dgemm_kernel.c
$(CC) $(CFLAGS_dgemm_kernel) -S $< -o $@
clean:
rm -f *.o *.s bench
```

Initial

```
/* dgemm_kernel.c */

void dgemm_kernel(
    int m,
    int n,
    int k,
    const double *A,
    const double *B,
    double *C)
{
    int i,j,p;
    for (j=0; j<n; ++j) {
        for (i=0; i<m; ++i) {
            double cij = 0.0;
            for (p=0; p<k; ++p) {
                cij += A[i + p*m] * B[p + j*k];
            }
            C[i + j*m] = cij;
        }
    }
}
```

Final

```
/* dgemm_kernel.c w/ Chatgpt */
void dgemm_kernel(
    int m,
    int n,
    int k,
    const double * restrict A,
    const double * restrict B,
    double * restrict C)
{
    int i, j, p;

    for (j = 0; j <= n-4; j += 4) { /* Don't indent co-equal loops. */
        for (i = 0; i <= m-4; i += 4) {

            double c00=0.0, c10=0.0, c20=0.0, c30=0.0;
            double c01=0.0, c11=0.0, c21=0.0, c31=0.0;
            double c02=0.0, c12=0.0, c22=0.0, c32=0.0;
            double c03=0.0, c13=0.0, c23=0.0, c33=0.0;

            for (p = 0; p < k; ++p) {

                const double a0 = A[(i+0) + p*m];
                const double a1 = A[(i+1) + p*m];
                const double a2 = A[(i+2) + p*m];
                const double a3 = A[(i+3) + p*m];

                const double b0 = B[p + (j+0)*k];
                const double b1 = B[p + (j+1)*k];
                const double b2 = B[p + (j+2)*k];
                const double b3 = B[p + (j+3)*k];

                c00 += a0*b0; c10 += a1*b0; c20 += a2*b0; c30 += a3*b0;
                c01 += a0*b1; c11 += a1*b1; c21 += a2*b1; c31 += a3*b1;
                c02 += a0*b2; c12 += a1*b2; c22 += a2*b2; c32 += a3*b2;
                c03 += a0*b3; c13 += a1*b3; c23 += a2*b3; c33 += a3*b3;
            }

            C[(i+0) + (j+0)*m] = c00;
            C[(i+1) + (j+0)*m] = c10;
            C[(i+2) + (j+0)*m] = c20;
            C[(i+3) + (j+0)*m] = c30;

            C[(i+0) + (j+1)*m] = c01;
            C[(i+1) + (j+1)*m] = c11;
            C[(i+2) + (j+1)*m] = c21;
            C[(i+3) + (j+1)*m] = c31;

            C[(i+0) + (j+2)*m] = c02;
            C[(i+1) + (j+2)*m] = c12;
            C[(i+2) + (j+2)*m] = c22;
            C[(i+3) + (j+2)*m] = c32;

            C[(i+0) + (j+3)*m] = c03;
            C[(i+1) + (j+3)*m] = c13;
            C[(i+2) + (j+3)*m] = c23;
            C[(i+3) + (j+3)*m] = c33;
        }
    }

    /* cleanup rows for this 4-column panel */
    for (; i < m; ++i) {
        double c0=0.0, c1=0.0, c2=0.0, c3=0.0;

        for (p = 0; p < k; ++p) {
            const double a = A[i + p*m];

            c0 += a * B[p + (j+0)*k];
            c1 += a * B[p + (j+1)*k];
            c2 += a * B[p + (j+2)*k];
            c3 += a * B[p + (j+3)*k];
        }

        C[i + (j+0)*m] = c0;
        C[i + (j+1)*m] = c1;
        C[i + (j+2)*m] = c2;
        C[i + (j+3)*m] = c3;
    }

    /* cleanup columns for mod(n,4) .neq. 0 */
    for (; j < n; ++j) {
        for (i = 0; i < m; ++i) {
            double cij = 0.0;

            for (p = 0; p < k; ++p) {
                cij += A[i + p*m] * B[p + j*k];
            }

            C[i + j*m] = cij;
        }
    }
}
```

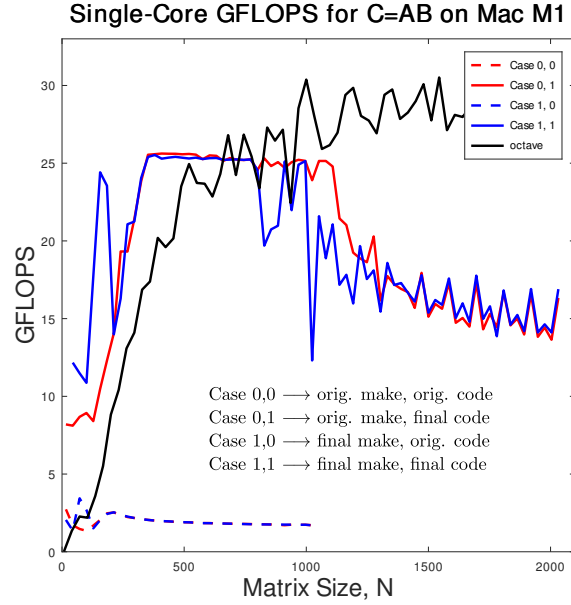


Figure 1: Composite performance results.

Figure 1 shows the result of Code initial/final coupled with Makfile initial/final (four combinations), along with the matlab code (running octave on my Mac M1), which has a baseline of $\sim 30+$ GFLOPS per core.

We see that the C code matters *much* more than the compiler choices, but those are nonetheless important. (We did not choose “bad” compiler flags in any case.)

Strikingly, the C code outperforms Octave in the low N values, which is of critical importances for distributed-memory parallelism, where we wish to run with as small an N as possible, *per rank*.

Moreover, the compiler flags *do matter* for low N , in this particular case.

We are able to get almost 25 GFLOPS at some smaller N values, but I would argue that these results are not stable and somewhat risky to rely on without a more detailed investigation.

Main conclusion is that C can outperform highly-tuned Octave code at the most important range of low N , and not so for the generally less-important range of large N . If large- N was of interest, we could continue to tune the C code.